

Incident Report: Samba Outage Caused by a Rogue find Process

Homelab incident-response simulation — nezuko (Ubuntu 26.04 LTS)

Field	Detail
Incident ID	INC-2026-06-15-001
Date	15 June 2026
System Affected	nezuko (Ubuntu 26.04 LTS)
Service Affected	Samba (smbd) — network shared file access
Severity	Critical
Status	Resolved
Author	Scott Pattison

Note — this is a simulated exercise carried out as part of homelab learning, written up in the format of a real-world incident postmortem to practise root-cause analysis and clear technical reporting.

Executive Summary

An unscoped `find / -name "*.log" -delete` command, run with root privileges, deleted log files across the filesystem — including Samba's active logs — and blocked staff from accessing shared network files. The offending process was identified and terminated, the deleted logs were restored from backup, and the Samba service was restarted to confirm recovery. Total impact: a full business day of lost access to shared files for office staff.

Timeline of Events

Elapsed	Event
T - 8h (approx.)	Rogue find command executed with root privileges, scope unrestricted from /.
T + 0	Office manager reports staff cannot access shared files on the server.
T + 5 min	Severity assessed as critical; office manager informed an investigation is underway.
T + 10 min	whoami confirms session identity; ps aux reveals a find process at 99% CPU, running ~8 hours.
T + 12 min	history confirms the exact command that was executed.
T + 15 min	Root cause identified: the find -delete sweep removed Samba's active log files.
T + 18 min	Process terminated (kill, then kill -9 after no response).

Elapsed	Event
T + 25 min	Deleted logs restored from the most recent available backup.
T + 30 min	Samba service restarted; shared file access verified as restored.
T + 35 min	Resolution and root cause communicated to the office manager.
T + 1h	Postmortem written up.

Root Cause Analysis

The command below was run as root, with no path scoping beyond the filesystem root and no dry run to preview matches first:

```
find / -name "*.log" -delete &
```

Because the search started at / rather than a specific directory such as /var/log, it recursively traversed the entire filesystem, deleting every matching *.log file it found — including Samba's active logs — while consuming 99% CPU for the roughly eight hours it ran undetected. The combination of sustained resource contention and the loss of Samba's log files left the service unable to operate correctly, which is what blocked staff from accessing shared files.

Resolution Steps

1. **Investigation** — identified the offending PID via:

```
ps aux  
history
```

2. **Immediate mitigation** — because the process was actively causing damage, it was stopped before full sign-off was sought. A standard kill was attempted first; the process did not respond, so it was force-killed:

```
kill <PID>          # no response  
kill -9 <PID>       # forced termination (SIGKILL)
```

3. **Data recovery** — the deleted logs were restored from the most recent available backup:

```
sudo tar -xzf /var/backups/logs-backup-20260614.tar.gz -C /var/log
```

4. **Service restart** — Samba was restarted so it could re-establish fresh, active log files.

5. **Verification** — confirmed shared files were accessible again and Samba was running cleanly before standing the incident down.

6. **Stakeholder communication** — the office manager was informed of the root cause, the action taken, and the follow-up recommendations below.

Lessons Learned & Follow-Up Actions

- **Scope destructive commands.** Never run `find -delete` from `/`. Target a specific directory and preview matches first by running the same `find` without `-delete` before committing to it.
- **Review junior staff permissions.** Restrict root/sudo access for staff who don't need it for their role, particularly for destructive operations.
- **Rotate the root password.** Treat the incident as a potential credential-handling issue and rotate root credentials as a precaution.
- **Tighten backup frequency.** The restored backup was already several hours stale by the time it was needed; more frequent log backups (or log shipping to a separate host) would shrink the recovery gap.
- **Add resource monitoring and alerting.** A process pinning 99% CPU for eight hours went unnoticed until a user-reported outage. Basic alerting on sustained high CPU/IO usage would have caught this long before staff were affected.
- **Enable sudo command auditing.** Logging sudo command history centrally would make tracing "who ran what, when" far faster in any future incident.